

# claude-windows / 902e27a6-e026-4b55-8c26-f2c47329cc44

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\subagents\workflows\wf\_c287c071-8e9\agent-a57cdf56a84824b9c.jsonl`
- SHA-256: `e41cda94ba298c92c9f1cae1b089b5e6f1c2b0d2c91bc4ad3fc0a1463133c572`
- Source modified: `2026-07-07T14:50:46+00:00`
- Imported at: `2026-07-08T16:00:01+00:00`
- project: `wf\_c287c071-8e9`
- session\_id: `902e27a6-e026-4b55-8c26-f2c47329cc44`

## Transcript

### 1. user (2026-07-07T14:48:49.999Z)

You are reviewing an uncommitted change in a git worktree at:  
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval  
Branch: fix/worker-skip-revalidation-on-dispatch (based on origin/master).

THE BUG BEING FIXED: On the cloud-serving path, the control-plane (backend/orchestration.py) runs validators and ONLY dispatches to the Cloud Run worker on PASS. The worker (backend/worker/\_\_main\_\_.py cmd\_infer) then REDUNDANTLY re-ran validation with its OWN config (different min\_blur\_threshold), rejecting a clip the control-plane already passed. Live failure 2026-07-07: passed at min\_blur\_threshold=8.0 on the control-plane, worker rejected at its default 20.0 ? GPU dispatch succeeded but the CUJ failed with worker rc=2.

THE FIX (skip-on-dispatch): a new ComputeJobSpec.skip\_validation bool (default False) ? backend/compute/cloud\_run.py emits a "--skip-validation" worker CLI flag when set ? backend/worker/\_\_main\_\_.py cmd\_infer honors it (skips validator\_registry.run\_all\_with\_context, sets val\_results=[], val\_context={}) ? backend/orchestration.py::\_maybe\_dispatch\_remote sets skip\_validation=True on every dispatch. The direct "worker infer" CLI (no control-plane) still validates by default. New tests in tests/test\_worker.py and tests/test\_api\_cloud\_dispatch.py.

Inspect the ACTUAL code ? do NOT trust this summary. Run: cd "C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval" && git --no-pager diff and read the surrounding functions (not just the diff hunks). The full test suite already passes (1779 passed), ruff + mypy are clean, coverage 85.42%. So do NOT report style nits or things the gate already covers. Hunt for REAL correctness/rollout/consumer defects the tests would miss. Only report a finding you can tie to a concrete failure scenario in THIS repo's code.

A reviewer (lens: tests) raised this finding about the change. ADVERSARIALLY verify it against the REAL code in C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval (read the files; run git grep / git diff). Default to REFUTED unless you can demonstrate the concrete failure in THIS repo's code. If real, give the minimal fix.

FINDING:  
severity=minor category=test-coverage  
file=tests/test\_compute\_backend.py line=405  
summary=The new \_dispatch\_remote propagation (worker/\_\_main\_\_.py:189, skip\_validation=bool(getattr(args, 'skip\_validation', False))) is never asserted -- test\_cmd\_infer\_dispatches\_when\_cloud\_target captures the spec but only checks video\_uri/segmenter, not skip\_validation.  
detail=The diff adds skip\_validation propagation to \_dispatch\_remote (the worker-CLI-with-cloud-target re-dispatch path). test\_cmd\_infer\_dispatches\_when\_cloud\_target (line 405) captures

seen['spec'] via a fake backend and asserts spec.video\_uri and spec.segmenter, but does not assert seen['spec'].skip\_validation at all -- neither the default-False case it currently exercises nor a True case (no test passes --skip-validation into this path). The propagation line is therefore uncovered. Impact is limited because a break here fails SAFE (the remote worker would re-validate rather than wrongly skip), and this path is an operator/debug path (the control-plane builds its spec directly and never calls \_dispatch\_remote). Add `assert seen['spec'].skip\_validation is False` to the existing test, plus a variant passing --skip-validation asserting True.

failure\_scenario=Someone removes or mistypes the skip\_validation propagation in \_dispatch\_remote (e.g. reverts the line). An operator running `worker infer --config <cloud-target> --skip-validation` would have the flag silently dropped and the remote worker would re-validate, contradicting the operator's explicit intent. No test fails because none inspects spec.skip\_validation on this path.

## 2. user (2026-07-07T14:48:56.992Z)

```
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
     frozen/hashable.
     config_overrides: tuple[str, ...] = ()
+    # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
+    plane) is
+    # the authoritative validation gate: it runs the SAME validators with ITS resolved config
+    and only
+    # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
+    and
+    # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
+    a clip
+    # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
+    plane)
+    # validating exactly as before.
+    skip_validation: bool = False
```

```
@dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
     ]
     if spec.segmenter:
         argv += ["--segmenter", spec.segmenter]
+    if spec.skip_validation:
+        # The control-plane already validated + only dispatches on PASS ? tell the worker
+        not to
+        # re-decode and re-validate (redundant, and its config may resolve different
+        thresholds).
+        argv.append("--skip-validation")
     for kv in (*spec.config_overrides, *self._container_overrides):
         argv += ["--set", kv]
```

```
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 5e9f510..79b4fed 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -568,11 +568,18 @@ def _maybe_dispatch_remote(
     prefetch = getattr(wasb_cfg, "prefetch_batches", None)
     if prefetch:
         overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
```

```

+ # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
+ # gate ? it already ran the validators with its resolved config just above (only a PASS
reaches
+ # here; a failure returned "failed" before dispatch). A worker re-run would decode the
video a
+ # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
+ # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
+ # then worker-rejected at its default 20.0 after a successful GPU dispatch).
spec = ComputeJobSpec(
    video_uri=req.video_path,
    out_uri=out_root,
    segmenter=seg_name,
    config_overrides=tuple(overrides),
+ skip_validation=True,
)
try:
    # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    out_uri=args.out_uri,
    segmenter=args.segmenter,
    config_overrides=tuple(args.set or ()),
+ skip_validation=bool(getattr(args, "skip_validation", False)),
)
try:
    result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
    video_id = compute_video_id(local_video)

- # 3. VALIDATE (same registry as the main CLI).
- val_results, val_context = validator_registry.run_all_with_context(
-     local_video, config
- )
+ # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+ # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+ # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+ # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
+ # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+ # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+ # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+ # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+ # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+ if getattr(args, "skip_validation", False):
+     print(
+         "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+         "gate and already validated this input."
+     )
+     val_results, val_context = [], {}
+ else:
+     val_results, val_context = validator_registry.run_all_with_context(

```

```

+         local_video, config
+     )
+     failures = [r for r in val_results if not r.passed]
+     db = Database(os.path.join(scratch, config.output.db_name))
+     db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
+         metavar="k=v",
+         help="config override, e.g. --set indexing.skip_ai_handoff=true",
+     )
+     pi.add_argument(
+         "--skip-validation",
+         action="store_true",
+         help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+         "validators with its resolved config and only dispatches on PASS, so re-validating here "
+         "is "
+         "redundant, decodes the video again, and can contradict the gate. Default-OFF: the "
+         "direct "
+         "`worker infer` CLI still validates.",
+     )
+     pi.set_defaults(func=cmd_infer)

+     pt = sub.add_parser(
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 8382234..19c37d9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -236,6 +236,22 @@ def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
+     assert "indexing.wasb.prefetch_batches" not in joined

+def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
+    """The control-plane is the AUTHORITATIVE validation gate: it validated + only dispatches
+    on PASS
+    (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on the
+    worker is
+    redundant, decodes the video a second time, and ? because the control-plane's validation
+    config
+    isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07: passed at
+    validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
+    dispatch MUST
+    carry --skip-validation, telling the worker to trust the gate. Regression guard for that
+    wiring."""
+    _db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    m._execute_processing(
+        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
+    )
+    assert "--skip-validation" in fake.calls[0]
+
+def test_cloud_failure_marks_failed_no_rows(cloud_env):
+    db, _bucket, monkeypatch = cloud_env
+    # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 8fb5e16..72b2514 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -97,6 +97,14 @@ def test_parser_infer_accepts_set_and_uris():
+     assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]

+def test_parser_infer_skip_validation_flag():
+     base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
+     # Default-OFF: the direct `worker infer` CLI still validates.

```

```

+     assert wmain.build_parser().parse_args(base).skip_validation is False
+     # The dispatcher passes it to disable the redundant worker re-validation.
+     assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
True
+
+
+     # ----- cmd_infer (local, mocked)

@@ -407,6 +415,43 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
    assert marker["returncode"] == 2 and marker["status"] == "failed"

+def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
+    """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run the
worker's
+    validators ? no second decode, and no chance of a threshold mismatch REJECTING a clip the
+    control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
+    run_all_with_context RAISE: a skipped run never touches it, so the job completes rc 0 with
a
+    status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no flag ? rc
2)."""
+    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
+
+    def _boom(*a, **k): # the validators must never be reached on the skip path
+        raise AssertionError("run_all_with_context must not run under --skip-validation")
+
+    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
+    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
+    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
+    video = tmp_path / "in.mp4"
+    video.write_bytes(b"FAKE")
+    out = tmp_path / "bucket"
+
+    rc = wmain.main(
+        [
+            "infer",
+            "--video-uri",
+            str(video),
+            "--out-uri",
+            str(out),
+            "--scratch",
+            str(tmp_path / "s"),
+            "--segmenter",
+            "wasb_hybrid",
+            "--skip-validation",
+        ]
+    )
+    assert rc == 0 # a clip that WOULD fail re-validation still succeeds ? the gate is trusted
+    report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
+    assert report["status"] == "success" and report["segment_count"] == 2
+
+
+    # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---

```

### 3. assistant (2026-07-07T14:49:02.047Z)

The diff is clear. Now let me examine the specific test claimed to be inadequate and the `\_dispatch\_remote` function it exercises.

### 4. user (2026-07-07T14:49:04.058Z)

```

360         msg = str(exc.value)
361         assert _NeverDoneClient.EXEC_NAME in msg
362         assert "cancel API unreachable" in msg # the cancel failure is surfaced, not

```

```

swallowed
363         assert "max_wait_sec" in msg # ...alongside the original timeout text
364
365
366     def test_poll_timeout_late_marker_rescues_result(monkeypatch):
367         """A slow-but-healthy worker that finishes JUST past the poll deadline
(max_wait_sec=1800 can
368         undercut the job's --task-timeout=3600) is rescued by the ONE final post-timeout
marker check:
369         the completed GPU result is ingested, nothing is cancelled, no timeout surfaces."""
370         from backend.compute import cloud_run as cr
371
372         exists_calls = {"n": 0}
373
374         class _FakeStore:
375             def exists(self, ref):
376                 exists_calls["n"] += 1
377                 return exists_calls["n"] >= 2 # absent for the poll, present on the final
check
378
379         monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
380
381         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
382             if dst is None:
383                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-late-"), "m.json")
384                 with open(dst, "w", encoding="utf-8") as f:
385                     f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
386                 return dst
387
388         monkeypatch.setattr(cr, "fetch", fake_fetch)
389         client = _NoopRunClient()
390         backend = cr.CloudRunCompute(
391             run_client=client, job_name="j", region="r", token="t", policy=_TIMEOUT_NOW
392         )
393         result = backend.run_infer(
394             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
395         )
396         assert result.returncode == 0
397         assert result.video_id == "vlate" # the late result is ingested, not wasted
398         assert client.cancelled == [] # a finished execution is never cancelled
399         assert exists_calls["n"] == 2 # exactly ONE extra post-timeout check
400
401
402     # ----- worker wiring: dispatch branch +
done-marker
403
404
405     def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
406         """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
407         from backend.compute.base import ComputeResult
408         from backend.worker import __main__ as wmain
409
410         seen = {}
411
412         class _FakeBackend:
413             def run_infer(self, spec):
414                 seen["spec"] = spec
415                 return ComputeResult(
416                     returncode=0, video_id="vidX", outputs_uri="gs://b/outputs/vidX/"
417                 )
418
419         monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
420         # cloud-serving startup checks require the AI-handoff key (AI_HANDOFF_KEY_MISSING is
fatal).

```

```

421 monkeypatch.setenv("GEMINI_API_KEY", "test-key")
422 cfg = tmp_path / "c.json"
423 cfg.write_text(
424     '{"deployment": {"profile": "cloud-serving"}, "storage": {"backend": "gcs"}}'
425 )
426 rc = wmain.cmd_infer(
427     wmain.build_parser().parse_args(
428         [
429             "infer",
430             "--video-uri",
431             "gs://b/videos/clip.mp4",
432             "--out-uri",
433             "gs://b",
434             "--config",
435             str(cfg),
436             "--segmenter",
437             "wasb_hybrid",
438         ]
439     )
440 )
441 assert rc == 0
442 assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
443 assert seen["spec"].segmenter == "wasb_hybrid"
444
445
446 def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
447     """The container side: cmd_infer with --done-uri writes a completion marker
448     (video_id + rc) so a
449     remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
450     AI."""
451     from backend.worker import __main__ as wmain
452
453     class _OK:
454         passed = True
455         validator_name = "fake"
456         message = "ok"
457
458         def model_dump(self):
459             return {"validator_name": "fake", "passed": True, "message": "ok"}
460
461     monkeypatch.setattr(

```

## 5. user (2026-07-07T14:49:04.916Z)

```

175     """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
176     once the
177     outputs are durably in the bucket (the backend bucket-polls the done-marker). The
178     dispatched
179     container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
180     re-dispatch.
181     A remote job that never writes a completion marker (a failed/hung container) makes
182     the bucket
183     poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
184     (remote dispatch
185     timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
186     escape."""
187     from backend.infrastructure.execution_policy import PolicyTimeout
188
189     spec = ComputeJobSpec(
190         video_uri=args.video_uri,
191         out_uri=args.out_uri,
192         segmenter=args.segmenter,
193         config_overrides=tuple(args.set or ()),
194         skip_validation=bool(getattr(args, "skip_validation", False)),

```

```

190         )
191     try:
192         result = compute.run_infer(spec)
193     except PolicyTimeout as e:
194         logger.warning(
195             "[worker] remote compute did not complete (no marker within budget): %s", e
196         )
197         return 4
198     logger.info(
199         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
200         result.video_id,
201         result.returncode,
202         result.outputs_uri,
203     )
204     return result.returncode
205
206
207 def _write_done_marker(
208     done_uri: str,
209     video_id: str,
210     returncode: int,
211     segment_count: int,
212     status: str,
213     storage_cfg: dict,
214     scratch: str,

```

## 6. assistant (2026-07-07T14:49:58.262Z)

Now let me verify the finding's factual claims: (1) whether any test asserts `skip\_validation` on the `\_dispatch\_remote` path, and (2) confirm the routing so I know whether line 189 is actually executed by the existing test.

## 7. user (2026-07-07T14:50:00.414Z)

```

C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_api_cloud_dispatch.py:128:
"""Make backend.compute.get_compute_backend (re-imported inside _maybe_dispatch_remote) build a
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_api_cloud_dispatch.py:245:
carry --skip-validation, telling the worker to trust the gate. Regression guard for that
wiring."""
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_api_cloud_dispatch.py:252:
assert "--skip-validation" in fake.calls[0]
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_main_module_budget.py:48:
"_maybe_dispatch_remote",
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:100:def
test_parser_infer_skip_validation_flag():
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:103:    assert
wmain.build_parser().parse_args(base).skip_validation is False
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:105:    assert
wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is True
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:418:def
test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:419:    """--
skip-validation (set by the dispatcher, the authoritative gate) must NOT run the worker's
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:427:
raise AssertionError("run_all_with_context must not run under --skip-validation")
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py:447:
"--skip-validation",

```

## 8. user (2026-07-07T14:50:01.767Z)

```

174: def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
177:     container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.
234: def cmd_infer(args: argparse.Namespace) -> int:

```

```
240:     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
243:     # dispatcher forces compute_target=local_gpu), so it never recursively re-dispatches.
246:     return _dispatch_remote(compute, args)
270:     # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
```

## 9. assistant (2026-07-07T14:50:33.553Z)

I now have everything I need to rule on this finding.

### Verification results

**\*\*Routing (confirms line 189 executes):\*\*** ``cmd_infer`` at ``backend/worker/__main__.py:246`` returns ``_dispatch_remote(compute, args)`` when ``compute_target`` is cloud. So ``test_cmd_infer_dispatches_when_cloud_target`` (test\_compute\_backend.py:405) DOES flow through ``_dispatch_remote``, and line 189 (``skip_validation=bool(getattr(args, "skip_validation", False))``) IS executed ? with the default-False case (``build_parser`` now sets ``args.skip_validation=False`` via ``store_true``). So the line is **\*\*covered\*\*** for coverage-gate purposes; the coverage floor (85.42%) is not affected by this at all.

**\*\*The assertion gap (the finding's factual core) is TRUE:\*\*** the test asserts only ``seen["spec"].video_uri`` (line 442) and ``seen["spec"].segmenter`` (line 443); it never inspects ``seen["spec"].skip_validation``. And searching all tests confirms no test asserts ``spec.skip_validation`` on the ``_dispatch_remote`` (worker-CLI-with-cloud-target) path:

- ``test_api_cloud_dispatch.py:252`` asserts ``--skip-validation`` in the argv ? but that exercises the ``orchestration`` ``_maybe_dispatch_remote`` ? ``CloudRunCompute`` argv path, a different code path.
- ``test_worker.py:103/105`` is a pure parser test.
- ``test_worker.py:418`` exercises the ``local`` validation-skip branch in ``cmd_infer``, not ``_dispatch_remote``.

**\*\*But this is not a reportable defect against the bar of this review.\*\*** The production code at line 189 is **\*\*correct\*\*** ? it propagates ``skip_validation`` faithfully. There is no current failure in this repo's code. The finding's own "failure scenario" is a ``hypothetical future regression`` (someone reverts/mistypes the line), and the finding itself concedes the impact is fail-safe (a dropped flag makes the remote worker ``re-validate``, never wrongly skip) and that this is an operator/debug-only path the control-plane never uses (the control-plane builds its ``ComputeJobSpec`` directly in ``orchestration._maybe_dispatch_remote``, which IS assertion-covered by ``test_cloud_dispatch_skips_worker_revalidation``).

The task's bar is explicit: report only REAL correctness/rollout/consumer defects tied to a concrete failure in the current code, default REFUTED otherwise, and don't flag coverage nits the gate already covers. A missing assertion on a fail-safe, operator-only path ? where the line is already executed and the code is correct ? does not clear that bar.

## 10. user (2026-07-07T14:50:46.392Z)

Structured output provided successfully